

LAUREA MAGISTRALE IN ARTIFICIAL INTELLIGENCE

Morphological Image Processing

A Sequential and OpenMP Parallelized Approach

Parallel Programming for Machine Learning 2025

Francesco Stocchi

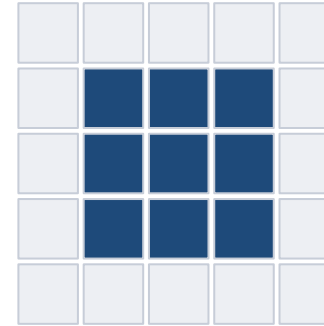
01

Introduction

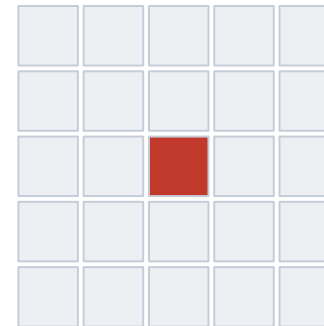
What is Morphological Image Processing?

- Theory for analyzing **geometric structures** within images
- Originally for **binary** images, later extended to **grayscale**
- Two fundamental primitive operators:
 - Erosion → shrinks foreground regions
 - Dilation → expands foreground regions
- Derived operators by composition:
 - Opening (erosion → dilation): removes small bright details
 - Closing (dilation → erosion): fills small dark gaps
- Every output pixel depends only on a **local neighborhood** defined by the structuring element

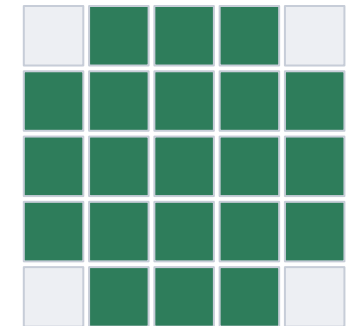
Original shape



Erosion (shrinks)



Dilation (grows)

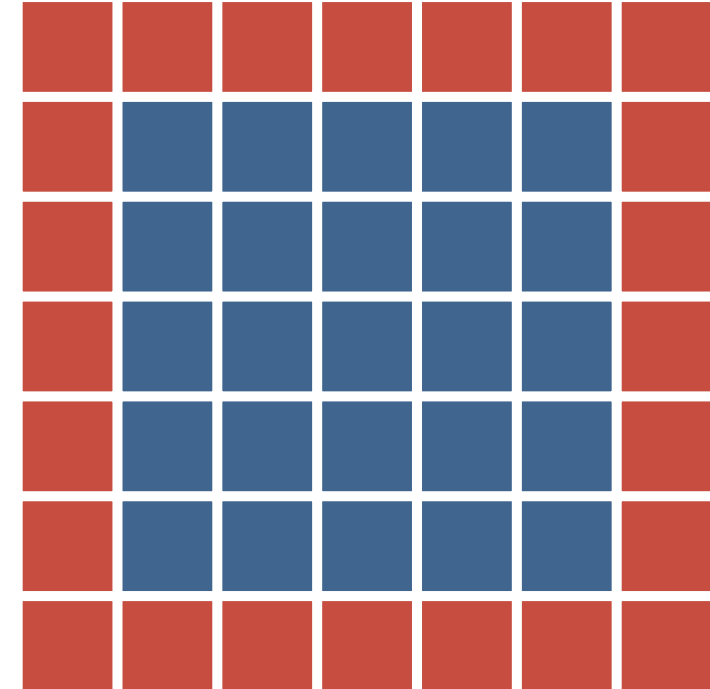


The Computational Challenge

- Morphological operators are naturally **data-parallel**
 - Each pixel's computation is independent of all others
 - Ideal candidate for OpenMP parallelization
- **Practical problem:** border pixels have an incomplete neighborhood
 - Requires explicit handling (e.g. clamping)
- Naive fix: a conditional branch in the innermost loop, for every pixel

```
if (ny>=0 && ny<H && nx>=0 && nx<W)
```
- This branching penalizes both **sequential** and **parallel** performance

Image grid: border vs. interior



 border pixels: need clamping

 interior pixels: vast majority

Project Goals

- 1 Implement the four morphological operators (erosion, dilation, opening, closing) in C++
- 2 Compare a sequential baseline against three progressively refined OpenMP variants
- 3 Isolate the impact of border handling on parallel performance
- 4 Measure speedup and efficiency across datasets of increasing resolution
- 5 Determine the optimal parallelization strategy for compute-bound, memory-regular kernels

Implementation Overview

Technologies used

Component	Choice
Language	C++ (performance-critical)
Parallelization	OpenMP: parallel for, collapse(2), tasks
Image loading	stb_image: header-only, dependency-free
Dataset	MS COCO 2017: 20,000 preprocessed images
Benchmarking	Custom runner with pixel-perfect verification + CSV output

Sequential Baseline

- Nested loops over rows (y) and columns (x)
- For every pixel: explicit bound check of the structuring element window
- **Problem:** the branch sits in the innermost loop for *every* pixel
 - Even for the vast majority of pixels far from any border!

Correct, but inefficient: both in sequential form and as the base for parallelization

erosion — sequential kernel

```
for (int y = 0; y < H; y++) {
    for (int x = 0; x < W; x++) {
        uint8_t min_val = 255;
        for (dy = -k/2; dy <= k/2; dy++)
            for (dx = -k/2; dx <= k/2; dx++) {
                int ny=y+dy, nx=x+dx;
                if (ny>=0 && ny<H && nx>=0
                    && nx<W)
                    min_val = std::min(min_val,
                                         in[ny*W+nx]);
            }
        out[y*W+x] = min_val;
    }
}
```

Variant 1: Naive Parallelization

V1: NAIVE

- Applies `#pragma omp parallel for collapse(2)` directly on the sequential loops
 - Keeps the per-pixel border branch inside the hot loop run by every thread
- Goal: isolate the “pure” benefit of multithreading alone
 - Result: early plateau

MAX SPEEDUP

~2.7x

Efficiency collapses to 0.09–0.15 at high thread counts

Variant 2: Border / Interior Separation

V2: BEST

- Explicit restructuring into two regions:
 - Borders: 4 thin strips, processed serially with explicit clamping
 - Interior: processed in parallel with direct, contiguous pointer access
- No bound-checking inside the dominant parallel loop

SPEEDUP RANGE

13.1–15.1×

Efficiency 0.74–1.05 — the interior is the vast majority of pixels

Variant 3: Parallel Border via Tasks

V3: TASKS

- Same border/interior split as Variant 2
- Adds OpenMP tasks for the 4 borders (one task per border)
- Interior is still processed with a standard parallel for
- Question: if parallel is good, is more parallel better?

AT HIGH THREAD COUNT

< 1x

Fixed task overhead eventually outweighs the small border workload

Correctness Verification & Dataset

Correctness verification

- Pixel-perfect comparison between sequential output and each parallel variant
- Cross-check between variants ($V1 \leftrightarrow V2$, $V1 \leftrightarrow V3$) to avoid masked bugs
- Any mismatch $\rightarrow$ offending image written to disk for inspection

All configurations passed the pixel-perfect check

Benchmark dataset

Dataset	Resolution	Images
Small	512×512	50
Medium	1024×1024	50
Large	2048×2048	50
X-Large	4096×4096	50

- Computational cost balanced across datasets (load-balancing $\propto 1/\text{size}^2$)
- Warm-up pass before every measurement to eliminate cold-start bias

02

Experimental Setup

Benchmark Configuration

Hardware

- CPU with 20 physical cores

Tested parameters

- Thread sweep: 1 (sequential) $\rightarrow$ 19 (max-1, leaving one core for the OS)
- Configurations: 4 operators $\times$ 4 datasets $\times$ 3 variants $\times$ N threads
- Timings averaged over multiple repeated runs

Methodological note: only the X-Large dataset (~ 10.6 s sequential) satisfies the ≥ 10 s rule for reliable timing. Smaller datasets show noisier curve.

20

physical cores

1 $\rightarrow$ 19

thread sweep

4 $\times$ 4 $\times$ 3

operators $\times$ datasets $\times$ variants

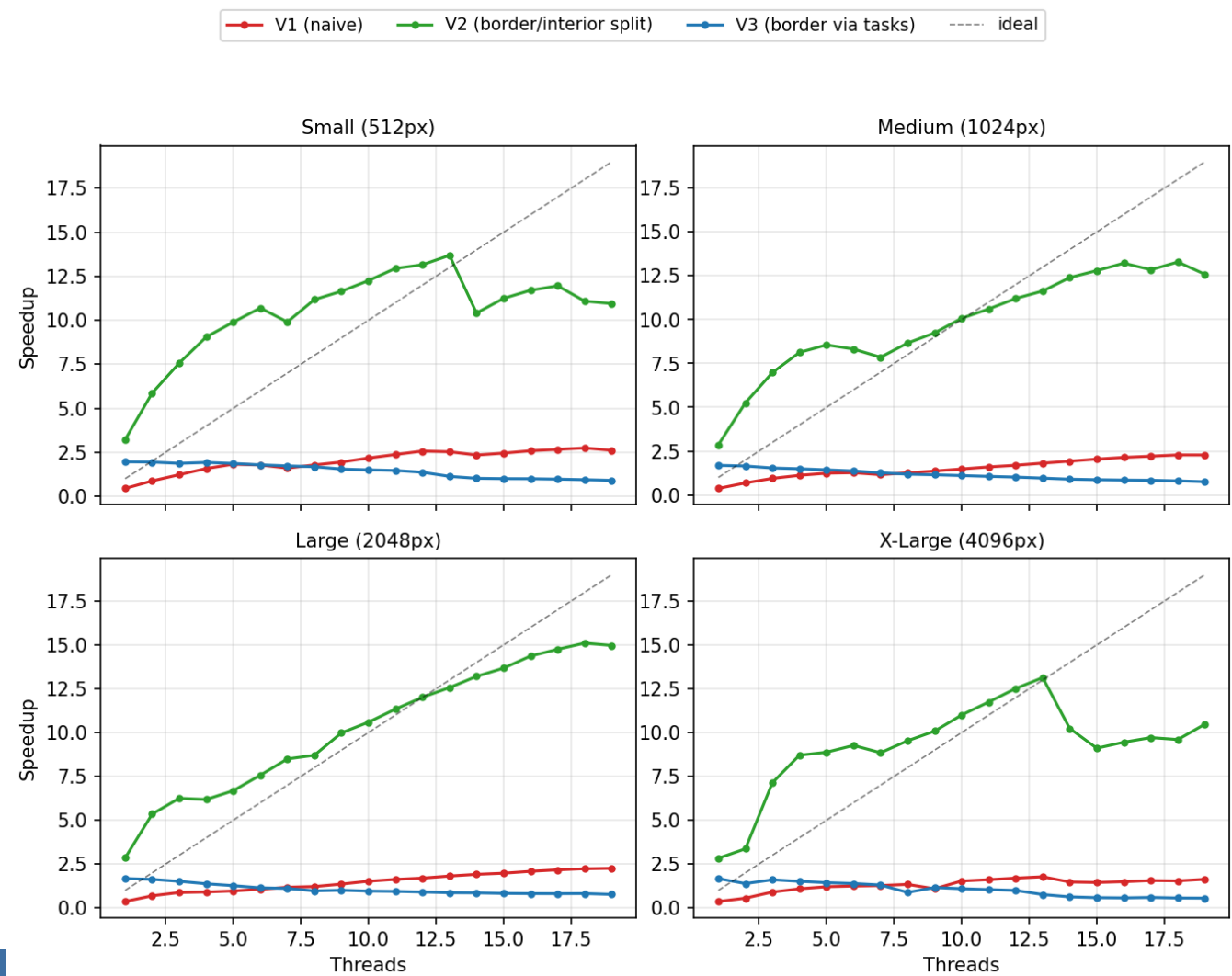
10.6 s

X-Large sequential runtime

03

Performance Results: Speedup

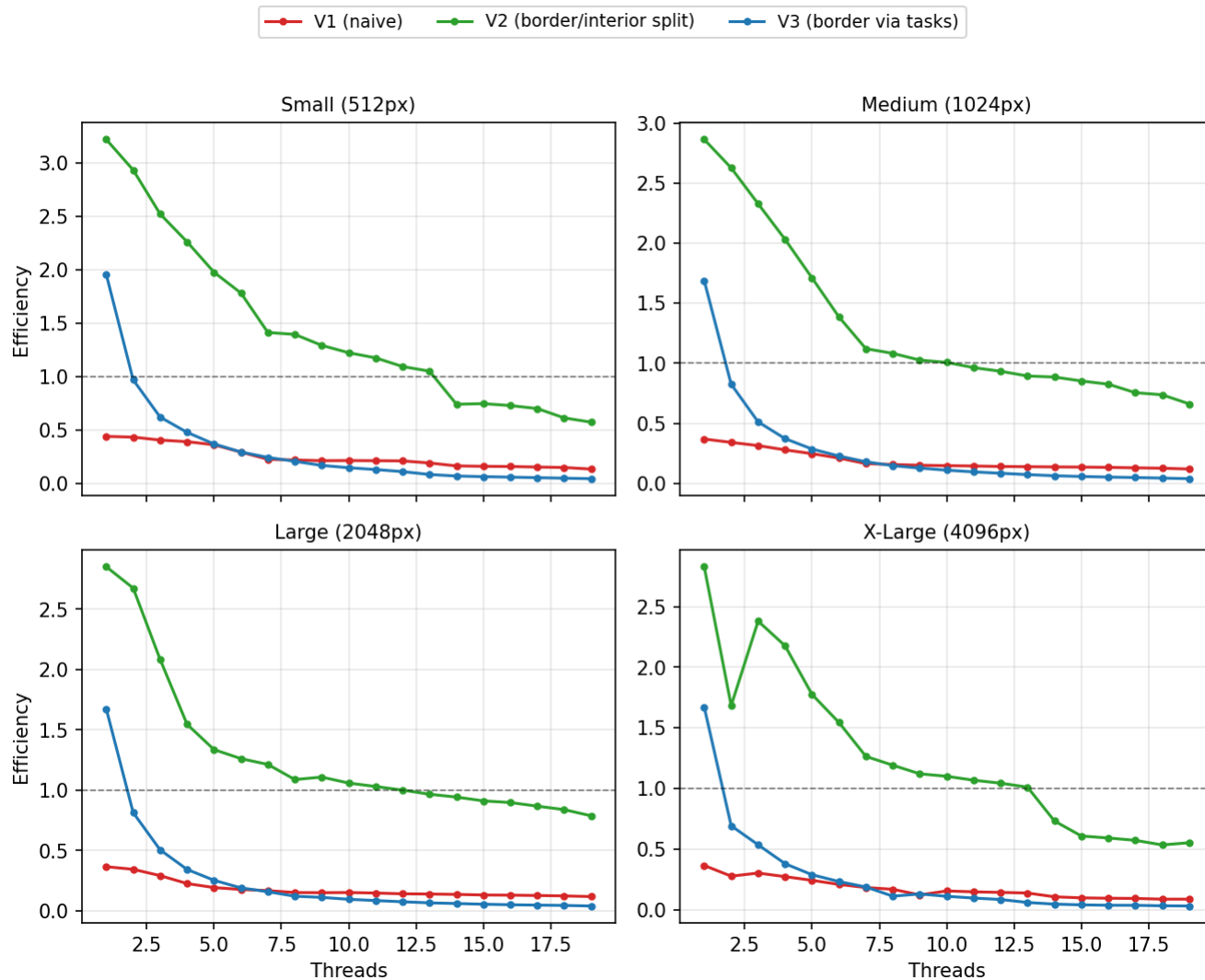
Speedup Comparison



Variant	Behavior
V1 (naive)	Plateau ~2.7×, efficiency drops to 0.09-0.15
V2 (border/interior)	Speedup 13.1×–15.1×, efficiency 0.74-1.05
V3 (tasks)	Degrades with more threads; speedup < 1 from 8+ threads on large datasets

Net result: V2 dominates across all dataset sizes

Parallel Efficiency



- V2 shows **superlinear speedup** at low thread counts (2–4 threads)

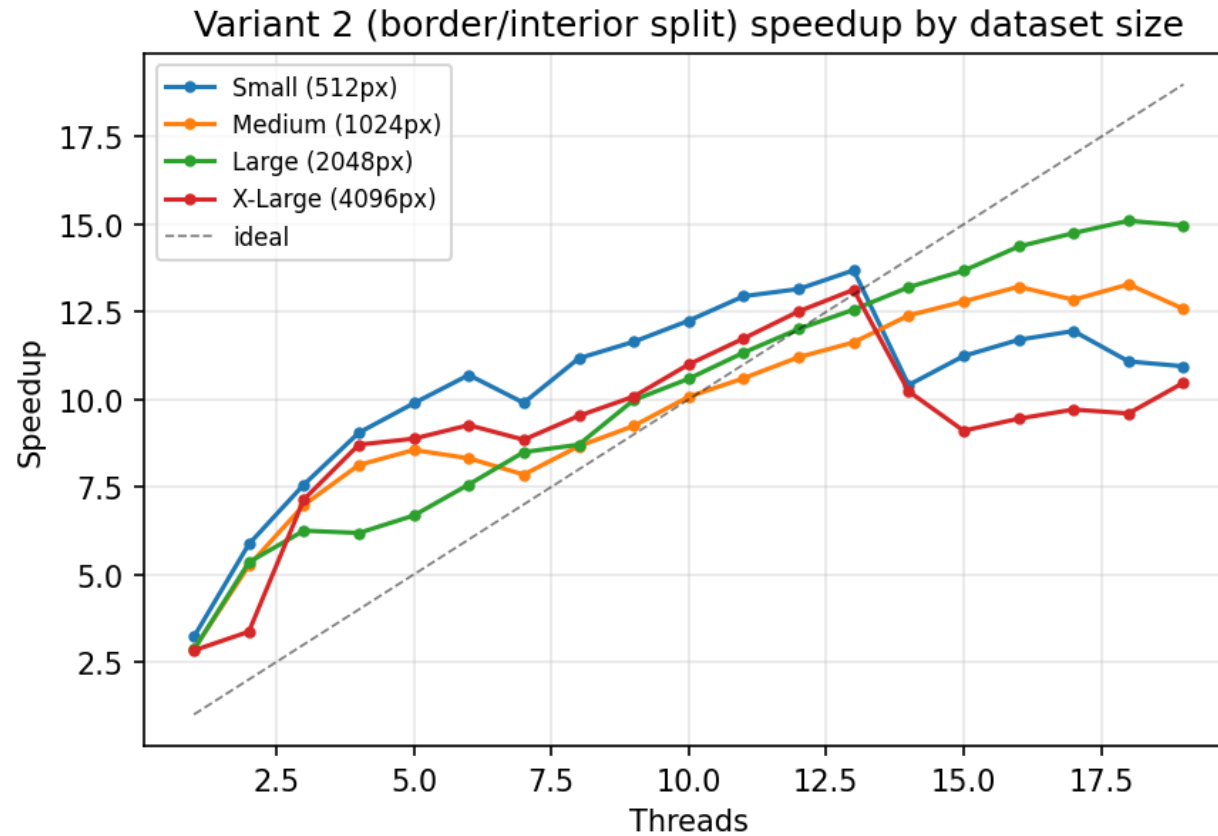
~2.9x

the ideal efficiency value

Cause: improved cache locality

- Large, contiguous chunks vs. sequential single-thread access
- This effect fades as chunks shrink with more threads

Variant 2 Deep Dive



Saturation beyond 15-18 threads:

- Scheduling overhead
- Cache refill at chunk boundaries
- Fewer rows per thread → less useful work
- Larger datasets (X-Large) show more stable, higher curves

Optimal configuration

13-18 threads on 20 physical cores

Reference: Figure 3, Speedup of Variant 2 (border/interior split) across the four dataset sizes.

Why Border Handling Matters

- V1 and V2 parallelize the exact same workload
- The crucial difference: V2 **removes the branch** from the inner loop

Quantitative impact

V1

~2.7×

V2

~15.1×

→ roughly 5-6× larger speedup from the optimized variant

- V3 confirms it: parallelizing a small, fixed workload (the borders) adds **constant overhead**
 - With many threads, overhead dominates → regression

TAKEAWAY

For compute-bound and memory-regular kernels, how you parallelize matters more than how many threads you use.

Key Results Summary

Dataset	Resolution	Seq. time	Speedup V2 (best)	Optimal threads
Small	512 px	~0.20 s	~11–13×	~13
Medium	1024 px	~0.50 s	~13×	~15
Large	2048 px	~2 s	~14–15×	~17
X-Large	4096 px	~10.6 s	~15.1×	~18

BEST PARALLEL EFFICIENCY

up to 1.05

(superlinear)

V2 is the winning strategy across every dataset size

Implications & Lessons Learned

1

Eliminating branching from the hot loop is the highest-impact optimization

V1 vs. V2: same workload, same thread count, 5× different result

2

OpenMP tasks are not “free”

Fixed overhead on a small workload → regression at high thread counts

3

Optimize the dominant portion of the kernel

Don't parallelize every region regardless of its size

4

Superlinear speedup is possible

Through improved cache locality (contiguous chunking vs. sequential access)

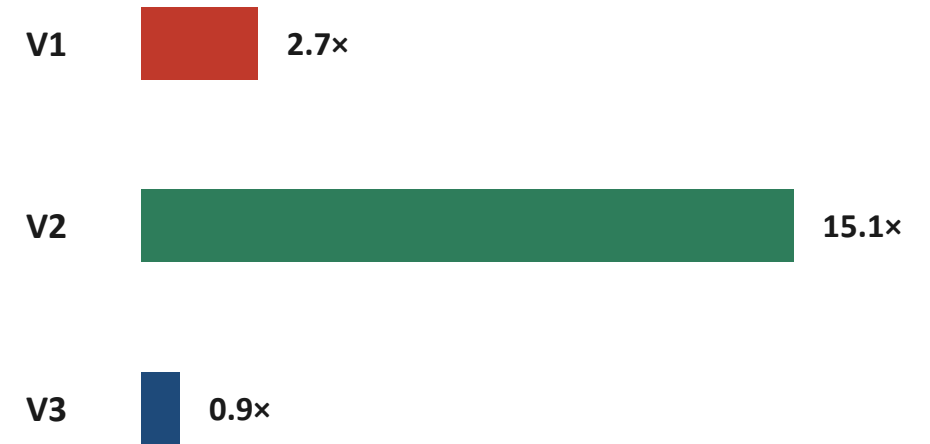
Conclusions

- The **border/interior separation (V2)** is the winning strategy
 - Speedup up to 15.1× with near-ideal efficiency
- **Naive border handling** drastically limits scalability
 - Regardless of the number of threads available

Future work

- Test structuring elements of different shapes (not only squares)

Speedup at a glance



15.1× peak speedup on X-Large images with Variant 2

Thank you for your attention

Francesco Stocchi

francesco.stocchi1@edu.unifi.it

[https://github.com/francescostocchi/Morphological_Image_Processing]

Questions?